

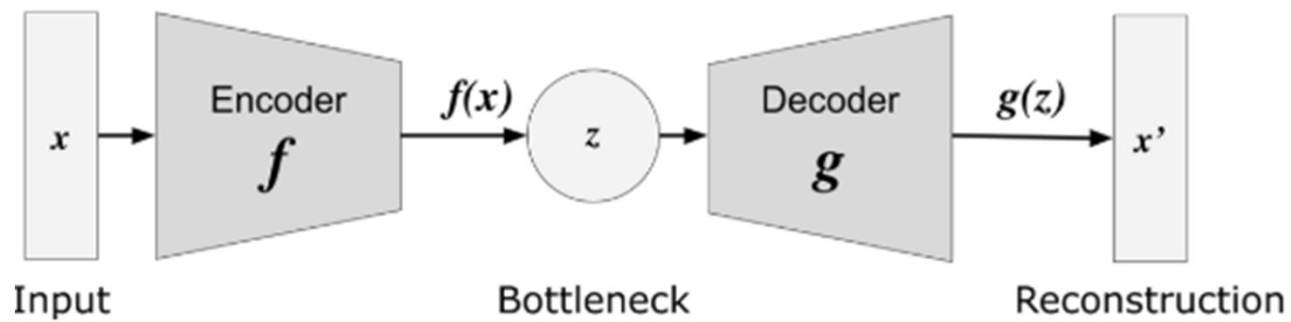
Module

Autoencoders, VAE, and

GANs

Moe Fadaee

Autoencoder Review



Bok, V. (2025). *Variational Autoencoders (VAEs)*. In *Generative AI in Computer Vision*. Manning Publications.

Autoencoders

What is an Autoencoder (**AE**)?

- A type of representation learning model.
- Not necessarily deep learning, but often used in deep learning with convolutional layers.
- Consists of an encoder (which compresses the input) and a decoder (which reconstructs the original input).
- Can be used for anomaly detection, denoising, and self-supervised learning.

Encoder

- The encoder is a neural network which maps input data to a latent space.
- **Output:** The encoder outputs a compact representation, typically as a vector or distribution (in the case of VAEs).
- **Training Objective:** In an autoencoder, the encoder is trained to produce a representation that the decoder can accurately reconstruct.

Mathematically an encoder can be represented as

$$f(x) = z$$

Decoder

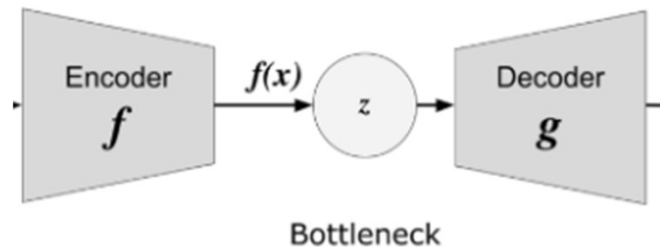
- The decoder is another network that takes what is in the latent space and tries to recreate it.
- **Output:** The decoder's output is a reconstruction or approximation of the original input data.
- **Training Objective:** The decoder is trained alongside the encoder in autoencoders.

Mathematically an decoder can be represented as

$$g(z) = x'$$

Bottleneck

- The bottleneck sits between the encoder and decoder
- Represents the latent space and contains a compressed representation



$f(x) = z \rightarrow$ data in bottleneck

$g(z) = x' \rightarrow$ reconstruction from bottleneck

Autoencoder wrap up

Autoencoder Framework

- x = Input
- $f(x)$ = Encoder
- $g(z)$ = Decoder
- $f(x) = z$ (Latent representation)
- $g(z) = x'$ (Reconstructed output)

Loss Function

- $\mathcal{L}(x, x')$ = Reconstruction Error
- Mean Squared Error (MSE):

$$\mathcal{L} = \frac{1}{N} \sum (x - x')^2$$

- Binary Cross-Entropy (BCE):

$$\mathcal{L} = -\frac{1}{N} \sum [x \log(x') + (1 - x) \log(1 - x')]$$

- Objective:

$$\min_{\theta} \mathcal{L}(x, x')$$

Training Process

1. Forward Pass:

- $z = f(x)$
- $x' = g(z)$

2. Compute Loss:

- $\mathcal{L}(x, x')$

3. Backpropagation:

- $\frac{\partial \mathcal{L}}{\partial \theta}$

4. Parameter Update:

- $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$

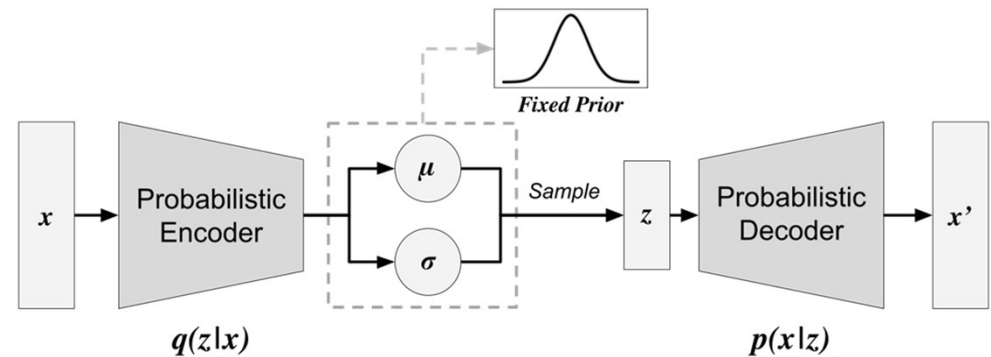
Variational Autoencoder

What is a Variational Autoencoder (**VAE**)?

- **Probabilistic Generative Model:** VAEs assume data is generated from an underlying probability distribution, enabling meaningful latent space representations.
- **Random Noise in Encoding:** Introduces stochasticity in the latent space, making reconstruction more robust and improving generalization.

Variational Autoencoder

- Unlike traditional autoencoders, VAEs generate a probability distribution for each encoding, allowing them to generate diverse outputs.
- Capable of generating new images by sampling from the latent space.



Bok, V. (2025). *Variational Autoencoders (VAEs)*. In *Generative AI in Computer Vision*. Manning Publications.

How does the VAE generate images?

- Latent Space Representation
- Sampling from the Latent Space
- Decoding the Sample

Latent Space Representation

- During training, the VAE learns a compressed representation of the input data (like images) in a lower-dimensional "latent space."
- Instead of encoding each input to a single fixed point, the VAE encodes it as a distribution (usually a Gaussian distribution).

```
1 z_mean, z_log_variance = encoder(input_img)
2 z = z_mean + exp(z_log_variance) * epsilon
3 reconstructed_img = decoder(z)
4 model = Model(input_img, reconstructed_img)
```

Sampling from the Latent Space

- Once the model is trained, we can generate new images by sampling points from this latent space.
- Draw samples from a Gaussian distribution centered at introducing random variations that allow the VAE to produce diverse outputs.
- **Issue:** Random sampling is non-differentiable.
- **Solution:** Reparameterization Trick

Reparameterization Trick

- Instead of directly sampling
$$z \sim \mathcal{N}(\mu, \sigma^2 I)$$
- We use the Reparameterization Trick:
$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$
- The Reparameterization Trick allows VAEs to learn through backpropagation despite random sampling.
- Instead of sampling directly, it shifts randomness to a separate variable, ensuring gradients can flow.

Decoding the Sample

- After sampling a point from the latent space, we pass it through the VAE's decoder, which reconstructs an image based on that latent representation.

```
decoder_h = Dense(256, activation='relu')
decoder_mean = Dense(784, activation='sigmoid')
h_decoded = decoder_h(z)
x_decoded_mean = decoder_mean(h_decoded)

vae = Model(input_img, x_decoded_mean)
```

VAE - Loss Components

VAE loss consists of two terms

- **Reconstruction Loss** - Ensures the decoded output x' is similar to the original input x
 - Typically **MSE** or **BCE**
- **KL Divergence Loss** - Regularizes the latent space by making the standard normal prior
 - Measures how much $q(z|x)$ deviates from $p(z)$

VAE - Complete Loss

Loss = Evidence lower bound (ELBO)

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

Reconstruction Loss (Data Fidelity):

- Measures similarity between input x and reconstructed x' .
- Common choices: Mean Squared Error (MSE), Binary Cross-Entropy (BCE).

$$\mathbb{E}_{q(z|x)}[\log p(x|z)]$$

KL Divergence Loss (Latent Space Regularization):

- Forces latent distribution $q(z|x)$ to match a prior $p(z)$

$$D_{KL}(q(z|x)||p(z)) = \frac{1}{2} \sum_{i=1}^d (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2)$$

VAE - Wrap up

VAE Framework

- Input: x
- Encoder:
 - $f(x) = (\mu, \sigma)$ (Latent distribution parameters)
- Latent Space Sampling:
 - $z \sim \mathcal{N}(\mu, \sigma^2 I)$
 - Reparameterization trick: $z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$
- Decoder:
 - $g(z) = x'$ (Reconstructed output)

Loss Function

- Evidence Lower Bound (ELBO):

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

- Reconstruction Loss:

$$\mathbb{E}_{q(z|x)}[\log p(x|z)]$$

- KL Divergence:

$$D_{KL}(q(z|x)||p(z)) = \frac{1}{2} \sum_{i=1}^d (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2)$$

- Objective:

$$\max_{\theta} \mathcal{L}$$

VAE Wrap up

Training Process

1. Forward Pass:

- $x \rightarrow \text{Encoder } f(x) \rightarrow (\mu, \sigma)$
- $z \sim \mathcal{N}(\mu, \sigma^2 I)$
- $x' = g(z)$

2. Compute Loss:

- $\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$

3. Backpropagation (Reparameterization Trick):

- $z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

4. Parameter Update:

- $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$

Generative Adversarial Network

- Consists of two models: a generator and a discriminator.
- The **generator** creates images or other data (starting from random noise).
- The **discriminator** evaluates the generated image and provides feedback (like a loss function), helping the generator improve over time.

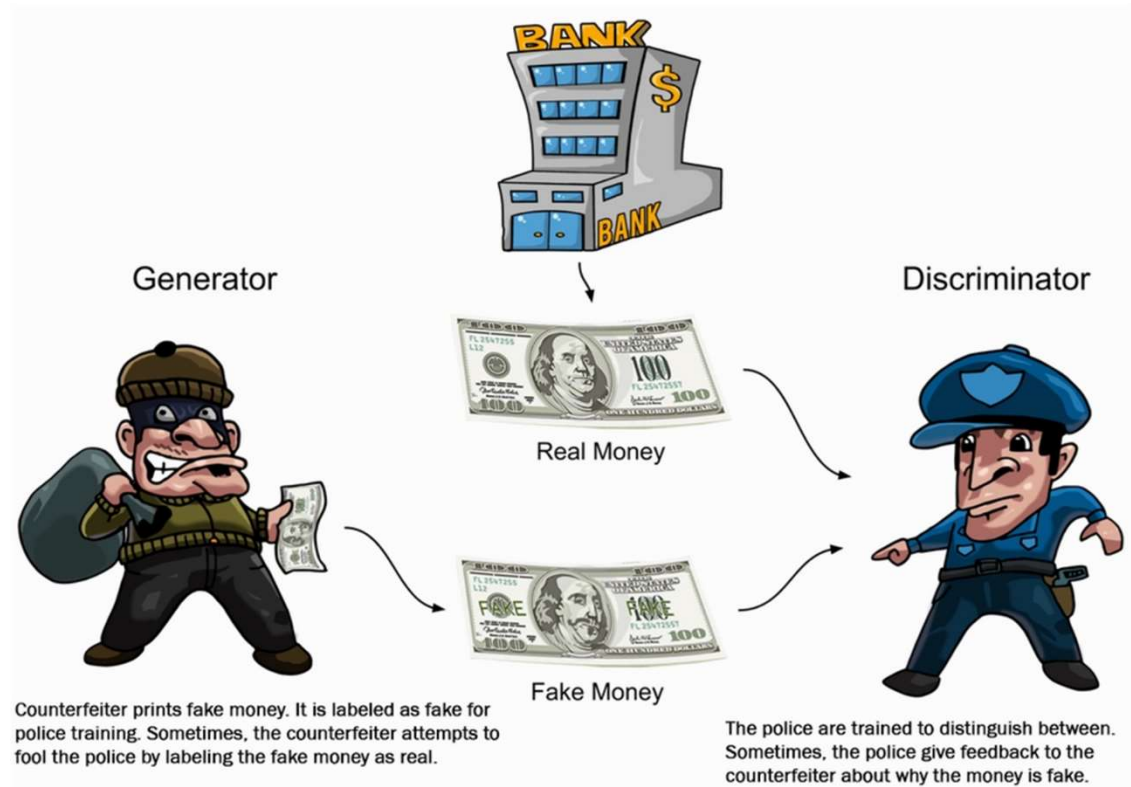
Generative Adversarial Network

Generator:

Role: A generator creates new data samples, typically starting from random noise or points in the latent space. Its purpose is to mimic the distribution of real data to produce realistic-looking outputs.

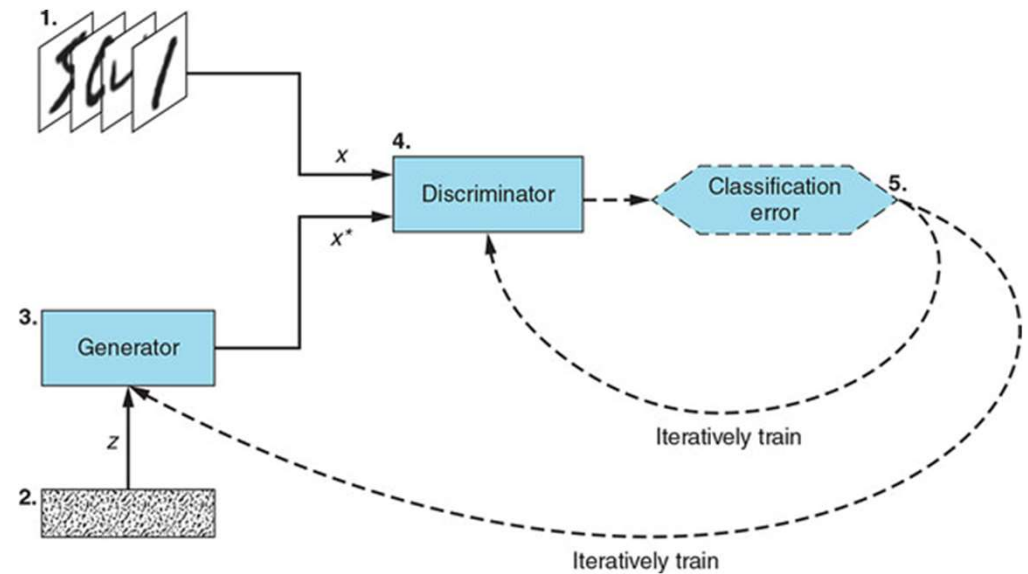
Discriminator:

Role: A discriminator is a classifier in GANs that distinguishes between "real" data (from the training set) and fake data (generated by the generator). It acts as a critic, pushing the generator to improve.



Generative Adversarial Network

- The discriminator evaluates the generated image and provides feedback (like a loss function), helping the generator improve over time.



Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.

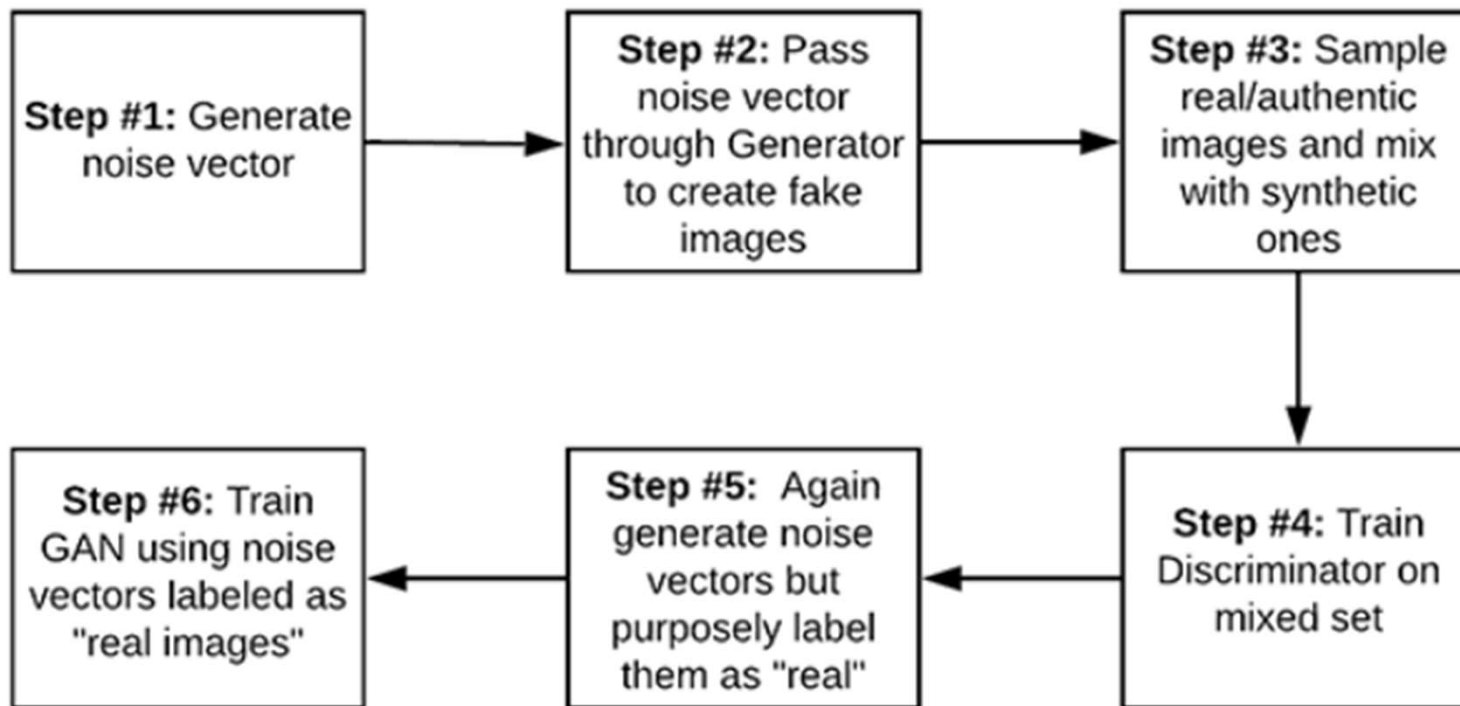
Generator

- **Output:** The generator's output is a "fake" sample (e.g., an image or other data) intended to look similar to the training data.
- **Training Objective:** In GANs, the generator's goal is to produce samples realistic enough to "fool" the discriminator. It receives feedback from the discriminator to improve its outputs.

Discriminator

- **Output:** The discriminator outputs a probability score (real or fake), usually between 0 and 1, indicating whether it thinks a sample is real or generated.
- **Training Objective:** The discriminator is trained to maximize its accuracy in distinguishing real from fake samples. It learns to recognize real samples' features and provide feedback to the generator.

Generative Adversarial Networks (GAN): the training process



Generative Adversarial Networks (GAN): the training process

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log \left(1 - D(G(z^{(i)})) \right) \right].$$

(1)

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(z^{(i)})) \right).$$

(2)

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Generative Adversarial Networks (GAN): the loss function

D = Discriminator

G = Generator

θ_d = Parameters of discriminators

θ_g = Parameters of generator

$P_z(z)$ = Input noise distribution

$P_{data}(x)$ = Original data distribution

$P_g(x)$ = Generated distribution

The binary cross-entropy loss:

$$L(\hat{y}, y) = [y \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Generative Adversarial Networks (GAN): the loss function

Now, when training the discriminator the label is $y=1$, real data and $\hat{y} = D(x)$:

$$L(D(x), 1) = \log(D(x))$$

When training the generator, the label is $y=0$, fake data and $\hat{y} = D(G(z))$:

$$L(D(G(z)), 0) = \log(1 - D(G(z)))$$

The discriminator should be able to correctly identifies the classes of the database, so the two above equations should be maximized, so final loss for Discriminator can be defined as:

$$L^D = \max[\log(D(x)) + \log(1 - D(G(z)))]$$

The generator is competing against the discriminator, G loss:

$$L^G = \min[\log(D(x)) + \log(1 - D(G(z)))]$$

Combine the 2 equations:

$$L = \min_G \max_D [\log(D(x)) + \log(1 - D(G(z)))]$$

What Makes Each Model Type Function?

- **Autoencoder:** Functions by minimizing reconstruction loss (difference between input and output). The encoder and decoder layers compress and reconstruct, respectively.
- **VAE:** Adds a probabilistic element to the latent space, allowing it to generate new samples by sampling from a learned distribution. This makes it generative.
- **GAN:** Relies on a unique training process where the generator and discriminator "compete" in a zero-sum game, refining each other through adversarial feedback.